

# CI/CD Pipeline Optimization Proposal

---

## CI/CD PIPELINE PERFORMANCE AND OPTIMIZATION PROPOSAL

Prepared By: DevOps Engineering Squad Date: August 18, 2026 Target Platform: GitLab Actions & Jenkins

**Problem Statement:** As our microservice architecture has scaled to over 80 independent services, our Continuous Integration / Continuous Deployment (CI/CD) pipelines have severely degraded in performance. The average build-to-deploy time for the core monolith has increased from 12 minutes in Q1 to 38 minutes currently. This delay creates friction for developers, slows down hotfix deployments, and consumes excessive cloud compute credits.

**Bottleneck Analysis:** 1. Sequential Test Execution: The integration test suite currently runs sequentially on a single runner instance. This step alone accounts for 22 minutes of the pipeline duration. 2. Inefficient Docker Builds: The Dockerfiles for our Node.js services are pulling and installing `npm` dependencies from scratch on every commit. Docker layer caching is not functioning due to the ordering of the `COPY` commands in the build definitions. 3. Artifact Bloat: The final container images for our frontend React applications average 1.2 GB because they include all development dependencies and source maps, dramatically slowing down image registry push/pull times.

### Proposed Optimizations:

- 1. Parallel Test Matrixing:** We will refactor the Jenkins pipeline to split the 5,000+ integration tests into 10 parallel execution chunks using matrix builds. This distributed approach will reduce test execution time from 22 minutes to under 4 minutes.
- 2. Dockerfile Refactoring and Multi-Stage Builds:** We will implement multi-stage Docker builds across all repositories. The first stage will handle dependency installation and compilation, while the final stage will only copy the compiled binary into a lightweight alpine or distroless base image. This is projected to reduce image sizes by 80%.
- 3. Dependency Caching:** We will introduce a localized Sonatype Nexus cache within the VPC to proxy npm and Maven packages. This will eliminate the latency of fetching packages from public internet registries during the build phase.

**Expected Outcome:** Implementing these changes over the next two sprints is projected to bring the median build-to-deploy time back under 8 minutes, saving an estimated 1,200 hours of developer wait time annually.